Received 28 February 2025; revised 3 May 2025; accepted 28 May 2025. Date of publication 5 June 2025; date of current version 18 July 2025.

Digital Object Identifier 10.1109/OJSSCS.2025.3577110

An Overview of AI Hardware Architectures and Silicon for 3-D Spatial Computing Systems

DONGSEOK IM, GWANGTAE PARK (Graduate Student Member, IEEE), JUNHA RYU (Graduate Student Member, IEEE), AND HOI-JUN YOO (Fellow, IEEE)

Department of Electrical Engineering, Korea Advanced Institute of Science and Technology, Daejoen 34141, South Korea

CORRESPONDING AUTHOR: D. IM (e-mail: dsim@kaist.ac.kr)

This work was supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) Grant funded by the Korea Government (MSIT, PIM Semiconductor Design Research Center) under Grant 2022-0-01170, and in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) through the AI Semiconductor Support Program to Nurture the Best Talents Grant funded by the Korean Government (MSIT) under Grant IITP-2025-RS-2023-00256472.

ABSTRACT As artificial intelligence (AI) advances, 3-D spatial computing has emerged as a key application in various fields. It interprets the 3-D space surrounding users and provides them with useful information. This article presents a survey of AI hardware architectures and silicon solutions for 3-D spatial computing systems. The survey categorizes five domains: 1) 3-D data capturing; 2) 3-D data analysis; 3) 3-D hand motion analysis; 4) simultaneous localization and mapping (SLAM); and 5) 3-D rendering. Each session analyzes design considerations for domain-specific accelerators. Finally, this article discusses a next-generation 3-D spatial computing platform that integrates various functions of 3-D spatial computing systems using AI technologies.

INDEX TERMS 3-D data analysis, 3-D data capturing, 3-D hand motion analysis, 3-D rendering, 3-D spatial computing, artificial intelligence (AI), simultaneous localization and mapping (SLAM).

I. INTRODUCTION

WITH the development of 3-D technology, 3-D spatial computing applications, such as autonomous driving, augmented reality (AR)/virtual reality (VR), and robot navigation, are successfully realized in modern industry. These applications perform computer vision algorithms on 3-D depth data complemented with a 2-D RGB image representing 3-D geometric information with colorful textures. Using 3-D RGB-D data, 3-D spatial computing systems understand not only 3-D positions and rotations of objects and surroundings but also rich color information, resulting in more natural communication with the user.

Unlike 2-D computer vision algorithms that search for features within the 2-D plane of an image, 3-D spatial computing systems analyze depth information through the z-axis in addition to the x and y dimensions. For example, object detection on 2-D images only generates 2-D rectangle bounding boxes, while object detection on 3-D RGB-D data extracts 3-D bounding boxes which capture both 3-D positions and 3-D rotations, representing six degrees of freedom (DoF). This richer spatial information enables more advanced functionalities for 3-D applications. However, the additional dimension in 3-D RGB-D data introduces significant complexity to 3-D computer vision algorithms, struggling to achieve sufficient accuracy in traditional hand-crafter 3-D computer vision algorithms. Thanks to artificial intelligence (AI) technologies, 3-D computer vision algorithms [1], [2] have surpassed the accuracy of hand-craft computer vision algorithms [3], [4], [5]. As a result, cutting-edge commercial products [6], [7], [8] have begun integrating AI-based 3-D computer vision algorithms into their vision pipelines, further demonstrating the remarkable impact of AI on 3-D spatial computing.

Major companies have developed specialized hardware and silicon to accelerate 3-D spatial computing systems on resource-constrained platforms. For example, Microsoft HoloLens 2 [9] has developed a hologram processing unit (HPU) that accelerates all computer vision algorithms, including hand tracking and simultaneous localization and mapping (SLAM) algorithms. Similarly, Apple Vision Pro [8] has designed an R1 silicon chip to perform low-latency 3-D data processing. However, their hardware

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

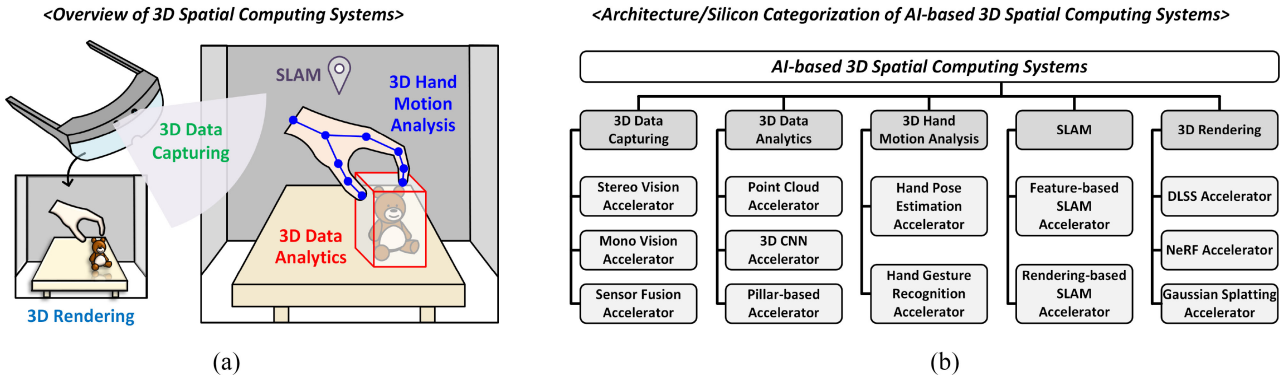

FIGURE 1. Scope of this article: (a) overview of 3-D spatial computing systems and (b) hardware architecture and silicon categorization of AI-based 3-D spatial computing systems.

designs still rely on traditional handcrafted computer vision algorithms in their vision pipelines. Consequently, there are opportunities to further optimize hardware and silicon by fully integrating AI algorithms.

Processing AI-based 3-D computer vision algorithms requires extra processing due to additional dimensions in 3-D RGB-D data. For this reason, conventional 2-D image accelerators cannot handle AI-based 3-D computer vision algorithms efficiently. For example, the spatial data tends to be randomly and sparsely distributed in 3-D space, and therefore, processing sparse data is a major bottleneck for 3-D spatial computing implementation. Finally, 3-D spatial computing accelerators have been developed to process 3-D data efficiently.

This article introduces hardware architectures and silicon solutions for AI-based 3-D spatial computing systems. It reviews previous hardware designs aimed at executing real-time AI-based computer vision algorithms on resource-constrained platforms. Moreover, it discusses future hardware designs for accelerating cutting-edge AI-based 3-D spatial computing systems efficiently.

II. CATEGORY OF AI-BASED 3-D SPATIAL COMPUTING

This article introduces AI-based 3-D spatial computing systems as shown in Fig. 1. The target domains include 3-D data capturing, 3-D data analytics, 3-D hand motion analysis, SLAM, and 3-D rendering. This categorization focuses on the prevalent 3-D applications and is considered as a representative application for 3-D spatial computing. *3-D data capturing* is a primitive process of 3-D spatial computing systems. It captures 3-D geometric information (x, y, z) with colorful textures (red, green, and blue) from an environment. Conventional 3-D spatial computing systems highly rely on 3-D scanners for 3-D data capturing. However, the main challenge of 3-D data capturing by 3-D scanners is high power consumption along with limited performance. For example, outdoor 3-D scanners [10] can capture long distances (>100 m) but they are limited in framerates (<20 fps) with high power consumption (>10 W). On the other hand, indoor 3-D scanners [11] limit the performance of 3-D data capturing in terms of sensing distance ranges (<10 m) and field of view (FoV) (<70°), but still consume sub-Watt (>3 W) of power, which is incapable of battery-limited edge platforms. Instead of using 3-D scanners, a 3-D data capturing system with AI technologies enhances the quality of 3-D RGB-D data from a low-cost RGB sensor. Therefore, low-power accelerators have been developed to acquire accurate and robust RGB-D data. Section III will describe three types of low-power AI-based 3-D data capturing systems: 1) stereo vision accelerator; 2) monocular vision accelerator; and 3) sensor fusion accelerator.

3-D data analytics are critical techniques for extracting semantic features from 3-D data to achieve accurate 3-D perception. They extract 3-D semantic features and enable 3-D shape classification, object detection, and segmentation by deriving meaningful semantic information. Current AI technologies outperform human capabilities in various perception tasks [12], [13], leading to the development of AI-based 3-D data analytics leveraging representations, such as point clouds, voxel grids, and pillars. However, AI-based 3-D data analytics require massive multiplier-and-accumulate (MAC) operations due to deep layers of AI algorithms. Moreover, processing unpredictably positioned 3-D data in 3-D space often results in inefficient hardware resource utilization on conventional GPU platforms. Hence, a high-throughput hardware architecture optimized for scattered 3-D data processing is necessary to perform AI-based 3-D data analytics efficiently. Section IV will illustrate high-throughput accelerators designed for 3-D data analytics, including point cloud-based neural network (PNN) accelerators, 3-D convolutional neural network (CNN) accelerators, and pillar-based neural network accelerators.

3-D hand motion analysis is a key technique for a user interface (UI) in many 3-D spatial computing platforms [6], [8], [9]. It analyzes human hands to estimate their positions and movements in 3-D space. 3-D hand motion analysis categorizes two primary algorithms based on purpose: 1) hand pose estimation and 2) hand gesture recognition. Hand pose estimation calculates the 3-D positions of approximately 21 joints in a human hand, enabling the modeling of a virtual hand that interacts with virtual objects in real time. On the other hand, hand gesture

recognition focuses on classifying specific hand shapes and movements into predefined meanings. Advanced AI algorithms accomplish high accuracy and robustness in both hand pose estimation and hand gesture recognition, making them essential for UI design in commercial AR/VR devices. Hand pose estimation involves intensive computations to accurately determine the 3-D positions of a human hand, making low-latency (<33 ms) implementation a critical requirement. In contrast, hand gesture recognition is a much simpler task, and widely exploited in ultralow-power Internet of Things (IoT) applications. As a result, hardware architectures are developed with specific target applications for 3-D hand motion analysis. Section V will discuss novel hardware architectures and silicon solutions developed for a 3-D hand motion analysis, including hand pose estimation accelerators and hand gesture recognition accelerators.

SLAM simultaneously generates both pose and map. It processes visual sensor data to determine the device's six DoF (6-DoF), referring to its position and orientation (tracking), while concurrently updating a 3-D map of the environment (mapping). These two processes are interdependent: tracking relies on the latest 3-D map, while mapping leverages historical camera poses, known as the trajectory. Traditional SLAM methods use handcrafted feature extraction and geometric pose estimation. They are computationally lightweight, requiring only <60 GOPS to achieve accurate camera poses with small (<0.1 W) power consumption [14]. Although effective in well-controlled environments, their performance often degrades in dynamic or texture-less settings. Consequently, parts—or even entire pipelines—of classical SLAM have been progressively replaced with AI-based models. While AI-based SLAM significantly improves accuracy and robustness, it introduces higher computational demands (300–13 000 GOPS or GFLOPS), posing a dual challenge: maintaining low power consumption while achieving high throughput. Section VI explores high-throughput SLAM hardware designs that optimize different AI-driven approaches, including feature-based and rendering-based SLAM accelerators.

3-D rendering is the task of generating photo-realistic 2-D images from the given 3-D models. It provides visually coherent images considering the user's viewpoint and environmental lighting. Since users perceive the 3-D scene through the rendered images, high-quality and real-time rendering is essential for a seamless experience. Key performance metrics for 3-D rendering include motion-to-photon latency, framerate, image quality and resolution, and energy efficiency. Motion-to-photon latency, which measures the time between a user's action and the corresponding display update, is critical for immersive experiences. The latency requirement is more stringent for AR/VR than for gaming PCs, as higher latency can lead to disorientation and nausea [15]. Higher framerate (>60 fps [16]) and image resolution further enhance immersion for both AR/VR and gaming PCs, with image quality commonly assessed using the structural similarity index measure (SSIM). On the other hand, energy efficiency is also a critical constraint, especially for AR/VR, due to the compact device size with the limited battery capacity.

Traditional methods have limitations in both 3-D model creation and rendering. For instance, photogrammetry struggles with feature matching on textureless or reflective surfaces and fails in creating complete models. In addition, ray tracing for rendering requires high computational cost to simulate light interaction, which becomes even more challenging in resource-constrained platforms. Recently, AI technologies have emerged to deliver immersive rendering experiences. Depending on the type of 3-D representation, AI can either assist traditional technologies or entirely replace them with novel approaches. Both cases require specialized hardware, as they demand numerous MAC operations or exhibit distinct operational characteristics. Section VII will discuss the hardware inefficiencies in AI-powered rendering and outline the three main types of hardware: 1) GPU with deep learning super sampling (DLSS); 2) neural radiance field (NeRF) accelerators; and 3) 3-D Gaussian splatting (3DGS) accelerators.

III. 3-D DATA CAPTURING ACCELERATORS

This section describes AI-based 3-D data capturing accelerators. Conventional 3-D spatial computing systems rely on 3-D scanners for RGB-D data acquisition. However, conventional 3-D scanners face performance limitations due to high power consumption and issues like missing pixels [17]. AI technologies address these inherent sensor limitations, achieving low-power and high-quality RGB-D data acquisition across various platforms. This section introduces AI-based 3-D data capturing accelerators, including stereo vision, monocular vision, and sensor fusion. Stereo vision exploits AI for robust feature extraction, while monocular vision directly infers depth from RGB data through AI. In the case of sensor fusion, different sensor outputs are combined through AI. Consequently, AI-based 3-D data capturing systems offer a promising solution for low-power and robust RGB-D data acquisition, and many hardware architectures and silicon designs have been proposed to support low-power and real-time AI-driven 3-D data capturing systems. Fig. 2 describes three approaches for 3-D data capturing, and Table 1 categorizes them with detailed methods.

A. AI-BASED STEREO VISION ACCELERATOR

Fig. 2(a) introduces stereo vision enhanced by AI technology. Stereo vision generates a depth map by computing the disparity between two images, comparing pixel differences between left and right views, and estimating cost functions. However, conventional stereo vision algorithms struggle with inaccurate depth estimation because of limited depth level and unintended depth smoothing [26]. To address these limitations, deep learning algorithms [27], [28] leverage neural networks to extract features from left and right images for cost functions and enhance feature matching through cost aggregation.

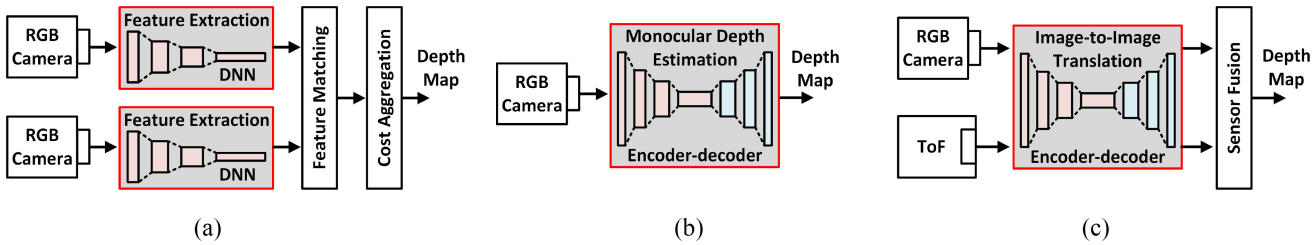

FIGURE 2. Three approaches for AI-based 3-D data capturing: (a) stereo vision, (b) monocular vision, and (c) sensor fusion.

TABLE 1. Categories of AI-based 3-D data capturing hardware.

Approach	Methods	Year	Implementation	Speed	Power	Application
Stereo Vision	Choi [18]	2018	ASIC	30 fps	0.00313 W	Hand Pose Estimation
Stereo Vision	Chen [19]	2020	FPGA	-	3 W	Mobile Robotics
Stereo Vision	Zhang [20]	2023	ASIC	-	1.096W	Mobile Robotics
Monocular Vision	Sada [21]	2020	FPGA	123.6 fps	0.3 W	Mobile Robotics
Monocular Vision	Im [22]	2023	ASIC	42.4 fps	0.0849 W	3D Object Detection
Monocular Vision (Training)	Park [23]	2022	FPGA	-	5.3 W	Mobile Robotics
Sensor Fusion	Im [24]	2022	ASIC	35.7 fps	0.194 W	3D Object Detection
Sensor Fusion	Sun [25]	2023	ASIC	34 fps	0.316 W	LiDAR System

AI-based stereo vision algorithms have adopted a deep learning algorithm for the feature extraction stage of a stereo vision pipeline. Nevertheless, the integration of the deep learning algorithm has led to a significant increase in computational demands, and to resolve this, new accelerators have been designed to incorporate new deep learning acceleration units within the existing stereo vision architecture. Choi et al. [18] adopted a CNN for feature extraction. A CNN processing element (PE) performs 2-D convolution by flattening filters and feature maps into 1-D data and utilizing shift MAC operations. This powerful CNN-based approach enables accurate hand pose estimation, and the silicon, fabricated using 65-nm technology, achieves 30-fps performance while consuming 3.13 mW of power. Chen et al. [19] and Zhang et al. [20] further minimized the complexity of CNN computations by adopting binary neural networks (BNNs) while maintaining high accuracy. Their implementations target for general mobile robotics and achieve 3 W (FPGA) and 1.096 W (ASIC) of power consumption, respectively, while delivering more than 100 fps.

B. AI-BASED MONOCULAR VISION ACCELERATOR

Monocular depth estimation acquires RGB-D data from a single image by computing monocular vision algorithms. This reduces sensor overhead compared to stereo vision systems. Recent advances in deep learning have achieved reliable depth accuracy in real-time speed [29], leading to extensive research on deep learning-based monocular depth estimation.

In contrast to AI-based stereo vision algorithms, AI-based monocular depth estimation uses standalone deep learning models that predict depth from a single RGB image. The deep learning model consists of an encoder–decoder network, which is larger models and harder to optimize than encoder-only stereo vision deep learning models. Therefore, significant effort is directed toward precise model optimization, such as pruning and quantization to preserve accuracy, and dedicated hardware accelerators are designed to efficiently process them. Sada et al. [21] developed a fast monocular depth estimation for a small-size robot on an FPGA. It focuses on SWaP (size, weight, and power)-optimized solutions, including pruning and quantization of CNN models. The system integrates a large number of MAC units to accelerate CNN computations, achieving 123.6 fps for 256×256 sized RGB-D image with a power consumption of only 0.3 W. Im et al. [22] proposed a silicon chip for monocular depth estimation. It adopted a bit-slice architecture to support variable bit precision in quantized CNNs, improving inference speed. The demonstration system shows 42.4 fps of speed with a power consumption of 84.9 mW at 80 MHz of clock frequency and successfully extracts 3-D bounding boxes on target objects from monocular RGB-D data. Park et al. [23] introduced an FPGA-based system for unsupervised learning-based monocular depth estimation for mobile robotics. This approach performs online training without requiring labeled datasets, making it effective in unstructured wild environments. The FPGA system utilized floating-point (FP) operations to handle the accurate computations of the deep learning model and introduced an efficient hardware architecture to enhance the performance. It integrated a partial switchable systolic array to accelerate separable convolution operations, achieving online training

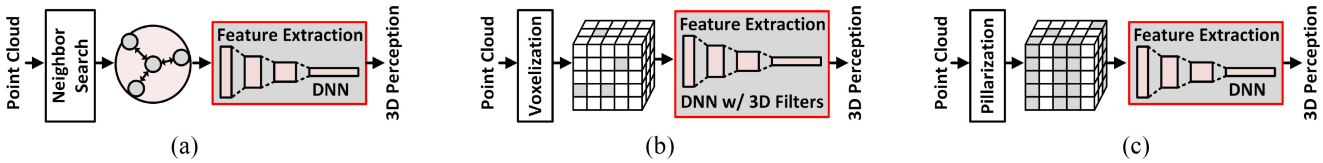

FIGURE 3. Three approaches for AI-based 3-D data analytics: (a) PNN, (b) 3-D CNN, and (c) pillar-based neural network.

demonstration with 5.3 W of power consumption and 54.4 GFLOPS of peak throughput.

C. AI-BASED SENSOR FUSION ACCELERATOR

A 3-D scanner (e.g., LiDAR) captures accurate depth data by emitting light and sensing the returning signal. Its reliable and stable depth acquisition is widely adopted in autonomous cars. However, 3-D scanners are expensive, and their spatial resolution and FoV are limited compared to RGB image sensors. Furthermore, extreme conditions in the environment, such as excessive darkness, brightness, and proximity, can result in failure to capture the returning light signal. To overcome these limitations, sensor fusion has been developed, combining multiple sensors to leverage their strengths.

Because sensor fusion systems require extra computations for integrating sensor data with the depth predictions from deep learning algorithms, minimizing the overhead of both the added sensors and computations is a key challenge. New hardware addresses this by carefully balancing the demands of the deep learning and sensor to find the most efficient operating point for acceleration. Im et al. [24] proposed a sensor fusion system-on-chip (SoC) combining outputs of a single camera and low-power time-of-flight (ToF) sensor for 3-D object detection. The system acquires RGB-D data through monocular vision algorithms and corrects its scales using true depth values from a lower-power ToF sensor. This SoC efficiently accelerates the complex sensor fusion algorithm by efficiently solving a conjugate-gradient method through band matrix encoding and decoding. This sensor fusion approach achieves 194 mW of power consumption and 35.7 fps of speed to process 19.2k RGB-D pixels. Sun et al. [25] developed a depth-completion system that upsamples low-resolution LiDAR or ToF depth maps to align with high-resolution RGB images. This LiDAR system consumes 316 mW of power and 34 fps of speed to generate VGA resolution of RGB-D data.

D. FUTURE DEVELOPMENT DIRECTIONS

Algorithms in 3-D spatial computing systems require diverse specifications of 3-D RGB-D data, including framerate, FoV, spatial resolution, and sensing distance range. For example, LiDAR is incapable of 3-D hand pose estimation which requires low-latency and short-distance sensing while an indirect ToF sensor is unsuitable for SLAM which requires long distance sensing with relatively low framerates for map generation. As a single 3-D scanner cannot accommodate all environmental and application requirements, various types of 3-D sensors are deployed to address these diverse demands. On the other hand, AI-based 3-D data capturing technologies mitigate the system overhead of traditional 3-D data capturing systems without relying on bulky 3-D scanners. By simply replacing software programs, these technologies can extract 3-D RGB-D data with varying qualities tailored to specific use cases. Moreover, monocular depth estimation on multicamera [30] produces high resolution and wide FoV depth map by applying image stitching and blending algorithms. As a result, with achievements in robust and accurate AI algorithms, flexible hardware architectures can efficiently execute diverse AI software algorithms in 3-D spatial computing systems and achieve reliable RGB-D data acquisition while adapting to surrounding environments.

IV. 3-D DATA ANALYTICS ACCELERATORS

This section describes 3-D data analytics accelerators. AI-based 3-D data analytics requires massive MAC operations due to the deep layers of AI models. In addition, processing 3-D data introduces higher complexity compared to 2-D image data due to its additional spatial dimension. For example, raw 3-D data is often irregularly scattered in 3-D space, leading to inefficient utilization of computing resources. Handling sparse 3-D data effectively is thus a critical factor in achieving high performance in 3-D data analytics. Accelerators tailored for point clouds, voxel girds, and pillars include specialized functional units to handle sparsity and improve throughput. As a result, various hardware architectures and silicons have been designed for accelerating 3-D data analytics in real time. Fig. 3 describes three approaches for 3-D data analytics, and Table 2 categorizes them along with their methods in detail.

A. POINT CLOUD-BASED NEURAL NETWORK ACCELERATOR

A point cloud is a fundamental 3-D data representation that consists of a set of 3-D points captured during the 3-D data capturing stage. To extract 3-D semantic features from a point cloud, PNNs are exploited that directly process raw point cloud data without refinement. Given the irregular distribution of 3-D points in 3-D space, these networks identify relationships among points and extract local features within neighborhoods. In specific, PointNet++ [47] samples center points from a point cloud and groups neighboring points around each center. The group points are processed through a multilayer perceptron (MLP) with shared weight parameters. The MLP outputs are pooled and aggregated

TABLE 2. Categories of AI-based 3-D data analytics hardware.

Approach	Methods	Year	Implementation	Speed	Power/Energy
Point Cloud-based	Feng and Tian [31]	2020	GPU	1.6× ↑	2.0× ↓
Point Cloud-based	Kim [32]	2021	ASIC	84.8 fps	0.2668 W
Point Cloud-based	Im [24]	2022	ASIC	130 fps	2.7 mJ/frame
Point Cloud-based	Feng [33]	2022	Simulator	1.9× ↑	1.5× ↓
Point Cloud-based	Chen [34]	2023	Simulator	12.1× ↑	-
Point Cloud-based	Becker [35]	2023	CPU Simulator	1.1× ↑	0.987× ↓
Point Cloud-based	Ying [36]	2023	Simulator	1.55× ↑	1.49× ↓
Point Cloud-based	Chen [37]	2023	Simulator	4.2× ↑	-
Point Cloud-based	Jung [38]	2024	ASIC	48.3 fps	0.3496 W
3D CNN-based	Zhou [39]	2023	ASIC	10 fps	0.6394 W
3D CNN-based	Cao [40]	2022	ASIC	7.2 fps	-
3D CNN-based	Sun and Feng [41], [42]	2023	ASIC	16.9 fps	0.1794 W
3D CNN-based	Feng [43], [44]	2024	ASIC	2.78 fps	0.264 W
Pillar-based	Lee [45]	2024	Simulator	4.1× ↑	349.3× ↓
Pillar-based	Lim [46]	2024	ASIC	44.2 fps	0.8555 W

through a max pooling layer where the maximal feature values within each group are selected to generate the final outputs.

However, sampling and grouping 3-D points bring about several hardware challenges in conventional accelerators. Irregular 3-D point distributions cause irregular memory access during sampling and grouping operations as well as increase MAC operations in the deep layers. Feng et al. [31] proposed an approximate delayed-aggregation algorithm that performs feature extraction before grouping. It reduces the size of input activation, alleviating MAC operations and memory footprints. Finally, its new algorithm achieves 1.6× speedup and 51.1% energy savings on a mobile GPU (Jetson TX2) with negligible accuracy loss (<1%). Kim et al. [32] introduced a block-based point cloud neural network accelerator to efficiently access a block-partitioned point cloud in a memory space. For high throughput at convolution layers, the accelerator supports input and output skipping operations. The custom silicon chip shows 84.8 fps of throughput and 266.8 mW of power consumption for a 3-D object detection task [48]. Im et al. [24] proposed a co-optimized software and hardware solution to efficiently accelerate PNNs. The accelerator incorporates shared operators for various sampling and grouping algorithms, including Poisson disk sampling-based uniform distance sampling [49], *k*-nearest neighbor (k-NN) search, and ball query, utilizing a unified point processing unit with a share of hardware resources. Moreover, the point feature reuse method was introduced to optimize the dense first layer of each set abstraction module [47], improving computational efficiency. The fabricated ASIC achieves 130 fps of throughput with 2.7 mJ/frame of energy consumption for a 3-D object detection task [48]. Feng et al. [33] solved the complexity of grouping operations through an approximate neighbor search algorithm coupled with a co-designed hardware architecture. The two-level kd-tree searches neighboring points efficiently, and selective bank conflict elision leverages SRAM bank conflicts by single-request processing. The architectural simulator shows 1.9× speedup and 1.5× energy reduction for PNN inference. Chen et al. [34] introduced a parallel octree-based k-NN algorithm for efficient grouping operations. This method facilitates conflict-free parallel searches, significantly improving throughput in point cloud neural networks. The architectural simulator integrated with high bandwidth memory (HBM) achieves 12.1× speedup over GPU implementations. Becker et al. [35] focused on compressing point clouds within kd-tree structures for ball query operations. By exploiting the spatial proximity of 3-D points, they leveraged similarities in the sign and exponent of FP representations to compress bit fields. Implemented as ISA extensions in a gem5-based out-of-order CPU, the simulator achieves 9.26% latency improvement on CPU benchmarks. Ying et al. [36] structured a raw point cloud using the Morton Code to sample and group 3-D points efficiently. Moreover, approximations of the sample stage and an index-based neighbor search stage minimize the end-to-end inference latency. The simulator achieves 1.55× speedup and 33% energy reduction over an edge GPU development board (NVIDIA AGX Xavier). Chen et al. [37] reduced competition and communication redundancies by exploiting geometric similarity among adjacent 3-D points. Low-bit differential values within group points reduce MAC computational costs, and the common neighbors across groups minimize communication costs during aggregation.

The architectural simulator shows 4.2× speedup over the GPU (NVIDIA Tesla A100). Jung et al. [38] designed a 28-nm chip to accelerate large-scale PNNs. It utilized a spherical coordinate-based bin search to enhance k-NN search efficiency and a point number-aware task scheduler to maximize hardware utilization. Finally, the chip achieves 48.3 fps and 349.6 mW of power consumption when processing 28.8k points.

B. 3-D CONVOLUTIONAL NEURAL NETWORK ACCELERATOR

3-D data analytics employ a voxel grid representation. A 3-D space is divided into regular grids, where 3-D points reside. Using this structured representation, a 3-D convolution filter in 3-D CNN [50] moves in three directions over 3-D voxel grids and extracts spatial and temporal features. However, the additional dimension in 3-D convolutions leads to significantly higher computational costs and numbers of parameters compared to 2-D convolutions [39]. Moreover, sparse point distributions in natural environments often cause sparse voxel inputs, which exacerbate inefficiencies during accelerating 3-D data analytics. To address this, Zhou et al. [39] proposed a 3-D CNN accelerator capable of skipping zero-value computations in both weights and input activations. This optimization achieves 19.5× faster speed than the nonskip baseline model. The 55-nm fabricated chip achieves a computation time of 0.1 s with 639.4 mW of power consumption for a C3D network [50].

To mitigate inefficient 3-D CNN computations, submanifold sparse convolution [51], [52], [53] focuses on generating convolution outputs only at valid voxel regions. Despite this efficiency, further optimization of sparse computations is still possible, leading to the development of specialized sparse CNN accelerators. Cao and Gu [40] presented a submanifold sparse convolution accelerator. It efficiently searches nonzero input activations and their corresponding weights through a hopping-index rule book that indicates spatial relations between nonzero input activations and weights while minimizing hardware costs. The 65-nm fabricated chip achieves 7.2 fps for a large-scale 3-D CNN model [53]. Sun et al. [41] and Feng et al. [42] presented a unified accelerator for both 2-D and 3-D CNNs. It efficiently performs gather and scatter operations in sparse point cloud using block-wise memory management. The processor presented a block-wise sparse storage format to support out-of-order off-chip memory allocation and continuous on-chip memory access. Moreover, the high throughput and reconfigurable core supports paralleled sparse neighbor searching for sparse convolution. This 28-nm fabricated chip achieves 16.9-fps speed with 179.4 mW of power consumption for a 3-D object detection task [54] with the large-scale KITTI dataset [55]. Feng et al. [43], [44] designed a scalable vision/point cloud fusion processor to efficiently handle the irregular operations in bird's eye view (BEV) fusion methods. The processor incorporates a content addressable memory (CAM)-based mapping unit to accelerate irregular submanifold sparse convolutions and supports chip-level scalability to meet increasing computational demands. The processor fabricated in 28-nm technology achieves 2.78 fps for a single chip and consumes 264 mW of power consumption.

C. PILLAR-BASED NEURAL NETWORK ACCELERATOR

A pillar-based neural network [56], [57] is an efficient 3-D object detector that simplifies a 3-D space into structured 2-D pillar representations. Pillars, which are a vertical columnar region in 3-D space, act as a bin to group 3-D points, enabling high performance with a light computational cost for 3-D object detection tasks. To efficiently process pillar-based neural networks, Lee et al. [45] introduced dynamic vector pruning that removes nonessential pillars while balancing accuracy and sparsity. Moreover, new sparse coordinate management hardware and sparsity-aware dataflow optimization efficiently compute vector sparse convolution operations. The architectural simulator achieves a 4.1× speedup with 349.3× energy savings compared to the GPU (RTX2080Ti) for a 3-D object detection task. Lim et al. [46] addressed the challenges related to external memory access and sparse characteristics in pillar-based neural networks and following backbone networks, respectively. The accelerator proposed a concept of a virtual pillar to minimize memory footprints through continuous processing. Moreover, a region-of-interest-based skipping method with quadtree-based workload management optimizes computation. The silicon chip is fabricated on 28-nm technology, achieving 44.2 fps with 855.5 mW of power consumption for a 3-D object detection task in the large-scale KITTI dataset [55].

D. FUTURE DEVELOPMENT DIRECTIONS

3-D data analytics algorithms vary significantly based on the chosen 3-D data representation. PNNs benefit from processing raw point cloud data, achieving high accuracy for small-scale point clouds. However, as the point cloud scales up, preprocessing brings about significant memory access and computational overhead in large-scale point clouds. 3-D CNNs represent point clouds as voxel grids, leveraging memory accesses and computation costs through voxel sparsity exploitation. However, voxel grid representation tends to lose fine details of small objects, degrading accuracy for small-scale point clouds in a large 3-D space. Pillar-based neural networks optimize voxel grids as structured pillars, but they only work for 3-D detection tasks in large-scale point clouds. As a result, different 3-D data representations are used based on specific target applications.

To support diverse 3-D point cloud analytics algorithms, efforts to unify hardware architectures have been researched. For example, Lin et al. [58] designed a unified accelerator that bridges mapping and matrix multiplication operations in PNNs and 3-D CNNs, efficiently executing gather and scatter operations for mapping operations and accelerates convolution operations for matrix multiplication operations. Moreover, sparsity in 3-D data remains a fundamental

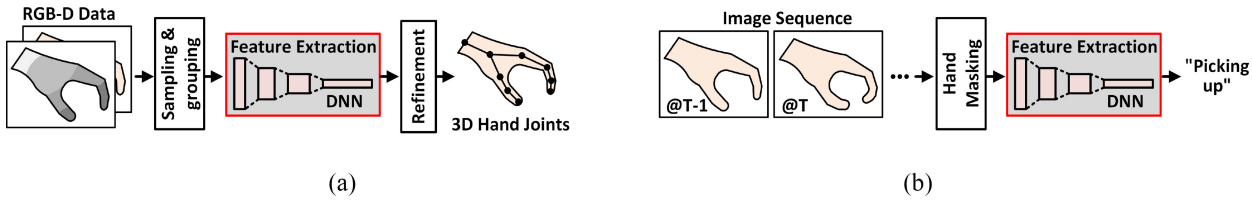

FIGURE 4. Two approaches for AI-based 3-D hand motion analysis: (a) hand pose estimation and (b) hand gesture recognition.

TABLE 3. Categories of AI-based 3-D hand motion analysis hardware.

Approach	Methods	Year	Implementation	Speed	Power
Hand Pose Estimation	Im [59]	2020	ASIC	224.7 fps	0.273 W
Hand Pose Estimation	Kim [60]	2024	FPGA	52 fps	2.217 W
Hand Gesture Recognition	Lu [61]	2021	ASIC	30 fps	0.000184 W
Hand Gesture Recognition	Lu [62]	2022	ASIC	60 fps	0.000181 W

challenge for accelerating diverse 3-D data analytics. Efficient hardware designs that exploit this sparsity will be critical for future 3-D data analytics. Finally, efforts should aim to develop versatile solutions that achieve high performance across different 3-D data representations, ensuring flexibility for a wide range of 3-D spatial computing applications.

V. 3-D HAND MOTION ANALYSIS ACCELERATORS

This section discusses AI-based 3-D hand motion analysis accelerators. AI-based 3-D hand motion analysis provides intuitive and precise recognition of hand movements and motions by extracting semantic features of humans hands. Given the high computational demands of AI models for 3-D hand motion analysis, achieving low latency and low power consumption is essential for real-time interactions on power-constrained platforms. In contrast to 3-D point cloud analysis accelerators described in Section IV, 3-D hand motion analysis accelerators address specific algorithms with highly stringent latency and power constraints. This has led to specialized hardware accelerators focused on aggressive algorithm optimization and pipelined hardware architecture particularly designed for these targeted algorithm pipelines. This section specifically explores for two main tasks in 3-D hand motion analysis: 1) 3-D hand pose estimation accelerators and 2) 3-D hand gesture recognition accelerators. Fig. 4 and Table 3 categorize and summarize them, and they showcase the innovative methods and optimizations employed to enhance the performance and energy efficiency of AI-powered 3-D hand motion analysis.

A. HAND POSE ESTIMATION ACCELERATOR

Hand pose estimation algorithms have been developed to accurately track finger joints in real time. Traditional model-based hand pose estimation algorithms [63], [64], [65] use a predefined hand model to estimate the 3-D hand pose. It relies on fitting the model to observed visual data by optimizing parameters, such as joint angles or bone lengths. In specific, Choi et al. [18] developed an iterated closest point (ICP)-particle swarm optimization (PSO) accelerator for high-speed hand pose estimation with sub-mW of power consumption. However, handcrafted model-based algorithms struggle with occlusions, complex poses, and noisy inputs.

AI-based data-driven hand pose estimation algorithms [66], [67], [68], [69] offer a solution to the robustness issue inherent in model-based algorithms. These techniques directly predict hand poses from input data by learning patterns from labeled datasets. To accelerate AI-driven hand pose estimation, Im et al. [59] proposed a low-latency hand pose estimation accelerator. The processor adopts the pipelined heterogeneous architecture to accelerate both sampling-grouping layers and convolution layers in a PNN, which is a major performance bottleneck. It proposed a window-based point sampling and grouping mechanism and large-scale (e.g., 16, 32, 64, and 128-to-1) max pooling prediction for low latency operations. The 65-nm fabricated chip achieves 4.45-ms latency for estimating 21 finger joints with 8.24 mm of an error. Kim et al. [60] proposed an FPGA-based real-time hand pose estimation system that adopts a parallel and pipelined structure to handle AI operations efficiently. Moreover, the FPGA system exploited dynamic on/off controls of PEs and designed a half-size shift buffer stack for improving energy efficiency. The FPGA system shows 52 fps with 2.217 W of power consumption for estimating 21 finger joints with 7.78 mm of an error.

B. HAND GESTURE RECOGNITION ACCELERATOR

Hand gesture recognition estimates predefined hand motions to interpret human commands and actions. AI technologies extract robust hand features from images, enabling high-accuracy recognition of both static and dynamic gestures. Lu et al. [61] designed a hand gesture accelerator that improves accuracy by integrating recognition results between CNN and a decision tree. The accelerator accelerates a lightweight edge CNN model by reusing data across PEs. Moreover, it adopted a pipelined hand localization unit

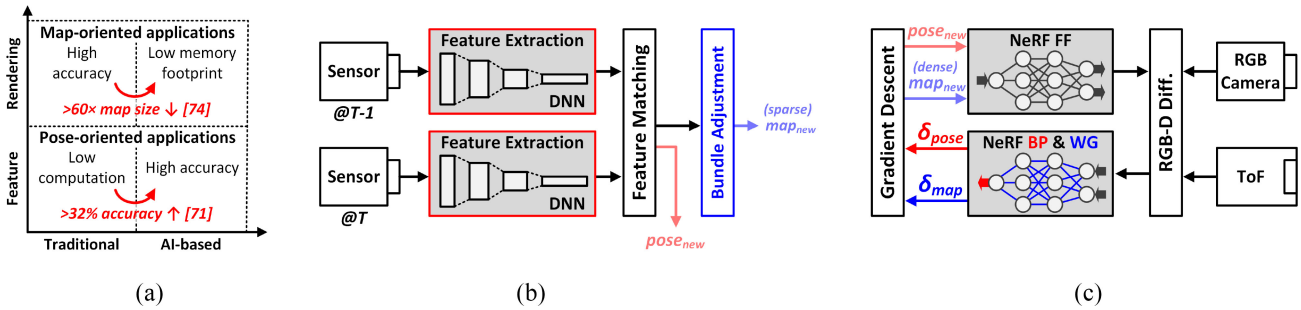

FIGURE 5. (a) Categorization of SLAM algorithms, including two AI-based SLAM algorithms: (b) feature-based SLAM and (c) rendering-based SLAM.

that tracks the hand locations in each frame with the minimum memory consumption, which resulted in low processing latency. To handle the temporal dimension, a sequence analyzer (SA) employs majority voting of Edge-CNN and DTC's static gesture recognition results over multiple frames, which improves robustness. The 65-nm chip achieves 30 fps and 184 μW of power consumption, recognizing 6 static hand gestures and 24 dynamic hand gestures. Lu et al. [62] introduced an upgraded hand gesture accelerator, leveraging depth and color fusion to mitigate segmentation challenges in complex backgrounds. Moreover, the pipelined row and column processing engines perform bi-directional convolution, improving robustness against hand rotation. The new chip shows 60 fps and consumes 181 μW of power consumption with the recognition of 6 static hand gestures, 20 dynamic hand gestures, and fingertip tracking.

C. FUTURE DEVELOPMENT DIRECTIONS

A human hand is a great tool for natural UIs, enabling the control of real and virtual objects in 3-D space. Depending on applications, various 3-D hand motion analysis algorithms have been developed to enhance user interaction. Hand pose estimation accelerators predict 3-D finger joint positions with low latency, allowing users to touch, translate, and rotate virtual objects realistically. On the other hand, hand gesture recognition accelerators focus on understanding hand movements and shapes, offering ultralow-power solutions for battery-limited platforms. These two types of algorithms can be integrated into a single 3-D spatial computing platform, such as an AR/VR device. In this setup, the device processes hand gesture inputs to trigger corresponding system functions and utilizes hand pose estimation for direct interaction with virtual objects. Additionally, the integration of large language models [70], [71], [72], [73] enables voice commands to control basic application functions, providing users with a more intuitive 3-D spatial computing experience. As a result, future hardware platforms are expected to support diverse UIs based on specific applications, combining AI-powered hand motion analysis and voice commands to improve the quality of user interaction.

VI. SLAM ACCELERATOR

This section introduces SLAM (or Visual SLAM), which generates tracking and mapping results from cameras without prior environmental knowledge. With advancements in AI, AI-based SLAM methods have gradually replaced traditional approaches, offering improved accuracy and robustness [79]. SLAM hardware must meet specific throughput demands. First, tracking throughput should match the device's operational frequency. For example, in smartphone-based AR, the 6-DoF refresh rate must align with the screen's refresh rate (30–120 fps). Mapping throughput requirement, in contrast, is lower, as updates occur periodically [80], [81] or at keyframes [82], [83]. However, mapping throughput and tracking accuracy are correlated [84], affecting another key metrics of SLAM system—robustness. For example, $<$ 10-cm accuracy is required to ensure low failure rate for indoor robots [85]. Absolute trajectory error (ATE) is the most commonly used metric for tracking. However, there does not exist a well-established metric for the mapping and often ignored in SLAM works. Several recent SLAM works utilize average distance or completion ratio [80] to evaluate the mapping robustness. In conclusion, the SLAM system should be carefully designed to satisfy throughput requirement while keeping reasonable robustness.

Fig. 5(a) illustrates a classification of SLAM algorithms along two key dimensions: 1) x-axis: feature-based or rendering-based and 2) y-axis: traditional or AI-based. Feature-based SLAM is primarily suited for pose-oriented applications, such as head tracking. Traditional methods offer low computational cost but are often limited in robustness. In contrast, AI-based feature extraction can identify more discriminative features per frame, leading to improved pose accuracy. Rendering-based SLAM focuses on constructing dense 3-D map, which are essential for map-oriented tasks, such as navigation or AR/VR interaction. Traditional approaches are accurate but require extensive memory cost to represent 3-D space in dense manner. In contrast, AI-based rendering SLAM compresses the scene for higher memory efficiency.

Table 4 summarizes AI-based SLAM hardware into feature-based and rendering-based approaches, discussed in the next section.

A. FEATURE-BASED SLAM ACCELERATOR

Fig. 5(b) outlines the overall process of feature-based AI SLAM. During tracking, AI models, such as CNNs

TABLE 4. Categories of AI-based SLAM hardware.

Approach	Methods	Year	Implementation	Tracking Algorithm	Mapping Algorithm	Speed	Power
Feature-based SLAM	Li [74]	2019	ASIC	CNN Feature Matching	Levenberg-Marquardt	80 fps	0.243 W
Feature-based SLAM	Jung [75]	2024	ASIC	PNN Feature Matching	Levenberg-Marquardt	48 fps	0.35 W
Feature-based SLAM	Li [76]	2024	ASIC	- (Off-chip)	Conjugate Gradient	-	-
Rendering-based SLAM	Park [77]	2024	ASIC	Neural Radiance Fields	Neural Radiance Fields	32.9 fps	0.3 W
Rendering-based SLAM	Wu [78]	2024	Simulator	3D Gaussian Splatting	- (Off-chip)	33.6 fps	9.4 W

or PNNs, extract features from input data provided by RGB or ToF camera. These features are then matched between consecutive frames, generating the 6-DoF pose. For mapping, bundle adjustment jointly optimizes camera poses and feature positions to produce a point cloud-based 3-D map. The nonlinear optimization required for this process is typically solved using approximations, such as the Levenberg–Marquardt or conjugate gradient methods [86]. In summary, feature-based AI SLAM is an assembly of heterogeneous algorithms.

AI-based features demonstrate improved robustness over handcrafted features—CNN feature matching [74] is 32% more accurate than SIFT [87] feature matching. However, incorporating AI into the SLAM pipeline significantly increases the operational intensity. Traditional SLAM processors, as in [14], require a throughput of less than 60 GOPS for real-time operation. In contrast, AI-based SLAM processors support peak throughput ranging from 300 to 1100 GOPS to accommodate real-time CNN or PNN inference [74], [75].

Since SLAM involves heterogeneous operations, feature-based AI SLAM processors optimize tracking and mapping separately. Li et al. [74] reduced tracking workload by exploiting temporal redundancy in feature matching. The mapping algorithm, employing the Levenberg–Marquardt method, is accelerated via a fixed-point quantized hardware unit with a hierarchical memory system. Evaluated on an autonomous driving dataset with a monocular camera, this ASIC achieved 80 fps at 0.243-W power consumption. Jung et al. [75] proposed a LiDAR SLAM processor with three hardware blocks. K-NN cluster minimizes external memory accesses via spatial and temporal locality, while the optimization cluster reduces latency via pipelining and parallelization. Each cluster integrates separate cores optimized for tracking and mapping. Finally, PNN cluster accelerates AI workloads through point-level task scheduling. The chip was evaluated on a LiDAR autonomous driving dataset, achieving 48 fps at 0.35-W power consumption. Li et al. [76] introduced an ASIC for mapping acceleration using the Conjugate Gradient method instead of Levenberg–Marquardt. Since Conjugate Gradient relies on iterative matrix multiplication, it enables most mapping operations to be accelerated through 16-bit FP computing-in-memory (CIM) arrays. However, such CIM arrays are suboptimal for tracking, where AI inference typically requires integer-level precision, necessitating a separate tracking accelerator.

B. RENDERING-BASED SLAM ACCELERATOR

Fig. 5(c) illustrates the flow of rendering-based AI SLAM. Here, 3-D map information is embedded in the parameters of a NeRF model, updated via DNN training. For both tracking and mapping, a NeRF feedforward (FF) operation renders an RGB-D image, and the absolute difference between the rendered and sensor images serves as the loss function. During tracking, backpropagation (BP) computes the 6-DoF pose gradient to minimize RGB-D loss. For mapping, BP and weight gradient updates (WG) optimize NeRF parameters. Rendering-based SLAM surpasses feature-based SLAM by handling featureless regions and reconstructing fully dense 3-D geometry [80].

Rendering-based traditional SLAM methods are memory-bound, primarily due to the need to represent 3-D scene using voxels. In contrast, rendering-based AI SLAM utilizes NeRF instead of voxel, achieving over 60× compression. However, it requires online training and thus imposes significant computational demands up to 13 TFLOPS [77]. In summary, the use of AI for rendering-based SLAM transitions the system bottleneck from memory to computing.

To address high computational demands, Park et al. [77] proposed a lightweight NeRF algorithm using a sparse mixture-of-experts (SMoE). Since tracking and mapping share the FF and BP stages, these are accelerated through a unified core. For the WG stage, a separate core with fewer MAC units is integrated, leveraging its higher sparsity. Additionally, a scene-aware sampling unit eliminates unnecessary workloads during preprocessing. The ASIC was evaluated using RGB-D images from a real-world indoor dataset, achieving a real-time throughput of 32.9 fps with 0.3-W power. Wu et al. [78] optimized SLAM using the 3DGS algorithm. While 3DGS improves tracking accuracy over NeRF, it is slower. To mitigate this, Wu et al. [78] proposed tracking with only informative image tiles. The accelerator was evaluated at the simulation level using a synthetic indoor dataset, demonstrating a throughput of 33.6 fps with 9.4-W power consumption.

C. FUTURE DEVELOPMENT DIRECTIONS

Despite advances in AI-based SLAM hardware, several challenges remain. One key issue is sensor power consumption.

TABLE 5. Categories of AI-based 3-D rendering hardware.

Approach	Reference	Year	Target Model	Implementation	Speed	Power
Ray Tracing + DLSS	NVIDIA [92]	2023	-	GPU	>200 fps	450 W (TDP)
Neural Radiance Field	Han [93]	2023	Implicit NeRF	ASIC	110 fps	0.899 W
Neural Radiance Field	Jo [94]	2023	Explicit NeRF (Tensor Decomposition)	ASIC	30.7 fps	0.13 W
Neural Radiance Field	Park [95]	2023	Explicit NeRF (Tensor Decomposition)	FPGA	33.6 fps	5.8 W
Neural Radiance Field	Ryu [96]	2024	Explicit NeRF (Hash)	ASIC	73.5 fps	0.728 W
Neural Radiance Field	Lee [97]	2023	Explicit NeRF (Hash)	Simulator	-	1.31 W
Neural Radiance Field	Li [91]	2023	Explicit NeRF (Hash)	Simulator	>30 fps	1.9 W
Neural Radiance Field	Yuan [98]	2024	Explicit NeRF (Hash)	FPGA	-	3.11 W
Neural Radiance Field	Fu [99]	2023	Generalizable NeRF	Simulator	24.9 fps	9.7 W
3D Gaussian Splatting	Lee [100]	2024	-	Simulator	91.2 fps	0.87 W
3D Gaussian Splatting	Feng [101]	2025	-	ASIC	373 fps	0.66 W
3D Gaussian Splatting	Song [102]	2025	-	ASIC	142.1 fps	0.62 W

For instance, the Intel RealSense L515 camera [11] consumes over 3 W to extract depth and RGB at VGA resolution–far exceeding the power budget of current SLAM ASICs in Table 4. A promising approach is fusing low-resolution ToF images with high-resolution AI-generated depth maps [24]. Therefore, future research must address the latency and computational overhead of sensor fusion.

Another key challenge is rendering-based SLAM's reliance on image quality. With low-cost cameras, its tracking accuracy is inferior to traditional SLAM. However, with high-quality image datasets, it can be over 10× more accurate than traditional SLAM [88]. This suggests that future SLAM hardware should evolve into SoCs with image enhancement feature.

Workload scheduling between tracking and mapping is another critical challenge. Although techniques like training loss prediction help reduce workload [89], they remain sensitive to factors, such as scene complexity and camera motion. While these methods maintain average throughput, they may struggle in worst-case scenarios. Future research should explore adaptive scheduling that dynamically allocates resources to ensure consistent throughput.

VII. 3-D RENDERING ACCELERATOR

This section introduces AI-powered 3-D rendering and its specialized hardware, categorizing approaches based on their target scenes. The first category is ray tracing combined with DLSS, which integrates AI into traditional rendering. Ray tracing has been extensively utilized for high-performance rendering in virtual environments, as it enables highly realistic lighting effects by simulating complex interactions of light rays with objects. However, achieving high-quality rendering requires a significant number of rays per pixel, which imposes computational and memory access burdens due to the irregular nature of ray traversal. To address these challenges, various software and hardware techniques leverage AI to improve rendering speed while maintaining or enhancing image quality. The second category includes NeRF and 3DGS, which tackle the challenges of photo-realistic 3-D modeling in real-world scenarios. These approaches reconstruct and render scenes from sparse real-world data by using machine learning to represent 3-D environments. For instance, NeRF and 3DGS train AI models using only 2-D ground-truth images, producing photo-realistic 3-D models without requiring manual design or expensive 3-D scanners. Although the operational pipeline of NeRF and 3DGS includes AI model training and inference, their characteristics differ from typical DNNs, causing hardware inefficiencies, such as bank conflicts, irregular off-chip memory access, and many MAC operations. The following sections discuss hardware architectures and silicon solutions for AI-based 3-D rendering, with Fig. 7 illustrating the DLSS, NeRF, and 3DGS algorithms, and Table 5 summarizing their hardware architectures and ASICs.

A. DEEP LEARNING SUPER SAMPLING

Fig. 7(a) presents an overview of NVIDIA DLSS [92], which significantly improves the quality of ray-traced scenes through ray reconstruction, super resolution, and frame generation. In traditional ray tracing, rays are cast from the camera's viewpoint to simulate light interaction with materials. However, the computational burden of this process is immense. For example, achieving reasonable rendering quality requires each primary ray to generate an average of ten secondary rays, and each pixel requires ten rays to minimize aliasing. Assuming 3.7M pixels for a single frame, maintaining a real-time framerate (30 fps) demands a processing throughput of 10 billion rays per second [16]. Such intensive computation results in prohibitive processing times, often taking minutes or hours even for offline rendering [92]. To mitigate these challenges, DLSS replaces computationally expensive ray casting operations with AI-powered ray

reconstruction, which generates high-quality pixels between sampled rays [92]. Besides, framerate can be boosted by rendering frames at a lower resolution and then upscaling them using a pretrained AI model, a technique known as super resolution. Moreover, frame generation also enhances framerate by using optical flow to create entirely new frames.

From a hardware implementation perspective, NVIDIA GPUs include ray tracing (RT) cores, CUDA cores, and tensor cores. To achieve ray-traced rendering, the RT cores accelerate bounding volume hierarchy (BVH) traversal to efficiently calculate ray-object intersections. Once the intersection is determined, geometry intersection and shading operations, which are highly parallel, are processed by CUDA cores. Finally, tensor cores perform the AI network inference for ray reconstruction or super resolution. In case of frame generation, an additional optical flow accelerator (OFA) is required (supported in NVIDIA's Ada Lovelace architecture [103]) to calculate the optical flow, capturing the direction and speed of moving pixels. As a result, DLSS can deliver an average 2.6× framerate improvement at 4K resolution on the RTX 40 Series with DLSS 3 [104].

B. NEURAL RADIANCE FIELD ACCELERATOR

NeRF encodes scene information into DNN parameters by iterative training. Once trained, users can generate rendered images from any arbitrary viewpoint via DNN inference. The NeRF pipeline consists of four main stages as illustrated in Fig. 7(b). First, during the sampling stage, 3-D coordinates are collected along the marching rays. Next, the encoding stage transforms these coordinates into high-dimensional feature vectors using either a sinusoidal function (implicit encoding) or feature embedding (explicit encoding). These interpolated feature vectors are then processed by an MLP to predict the color and density of each sample. Finally, volume rendering accumulates the final pixel color using a weighted sum operation. Since various NeRF variants have been developed, including implicit and explicit representations, distinct hardware bottlenecks have emerged due to the different inherent operational characteristics as illustrated in Fig. 6. To address these challenges, we introduce NeRF accelerators, discussed in the following order: the large computational workload inherent in implicit NeRF, inefficiencies in interpolation for explicit NeRF, and irregular memory access patterns common to both explicit and generalizable NeRF.

The amount of computation of NeRF is proportional to the number of samples, many recent NeRF accelerators focus on reducing the number of samples. This is especially critical for implicit NeRF [90] models which have more than ten MLP layers. Han et al. [93] proposed three techniques to reduce the number of samples for implicit NeRF acceleration: 1) skipping samples in empty space; 2) removing redundant samples from the previous frame; and 3) skipping invisible samples located backward of each ray. This work introduced heterogeneous DNN engines (1-D and 2-D) to efficiently exploit inherent sparsity. With dedicated hardware units, the processor achieved 110 fps of rendering at 899 mW power consumption.

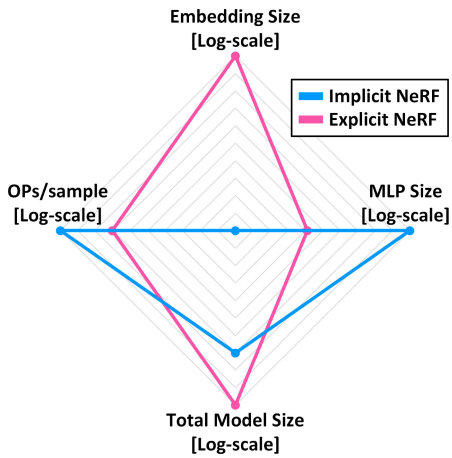

FIGURE 6. Characteristics (normalized number of operations and memory footprint) of NeRF variants, implicit [90] and explicit NeRF [91].

As NeRF algorithms for fast rendering have evolved, the explicit NeRF models have emerged. These models encode scenes using feature embedding and shallow MLP (around five layers), instead of encoding scene information solely in MLP parameters as implicit NeRF. However, explicit NeRF still shows low framerate rendering on edge devices due to inefficient memory access. To address this, Jo et al. [94] proposed an in-memory-computing-based explicit NeRF accelerator to reduce the read access power of the encoding stage. It achieved 30.7 fps while consuming 129.8 mW of power. Park et al. [95] also tackled large interpolation computations in explicit NeRF, and resolved it with pruning and early stopping techniques. Implemented on an Intel FPGA platform (Cyclone V), it achieved 33.6 fps of throughput with 5.8 W of power consumption. While Jo et al. [94] and Park et al. [95] accelerated the explicit model using tensor decomposition [105], recent NeRF accelerators have focused on hash-based embedding [106]. Hash-based NeRF uses hash function to retrieve the corresponding feature vector stored in the hash table. Ryu et al. [96] addressed irregular memory access caused by hash function, which led to large external memory access (EMA) and throughput degradation due to bank conflicts. This work suggested segmented hashing to divide 3-D scene into sub-blocks, reducing memory access by fetching and storing data at sub-block granularity. The proposed silicon chip accelerated both 3-D modeling and rendering, achieving 73.5 fps for rendering (728.4 mW), and 18.5 mJ/iter of energy consumption for modeling. Lee et al. [97] similarly addressed irregular memory access by implementing restricted hashing that partitions the hash embedding into small chunks and loads only a portion of the hash table. This approach reduced off-chip memory access and enabled pipelined operation between encoding and MLP operation. The architectural simulation showed 9.17× speedup in rendering compared to NVIDIA Jetson Xavier NX [107], with 1.31 W of power

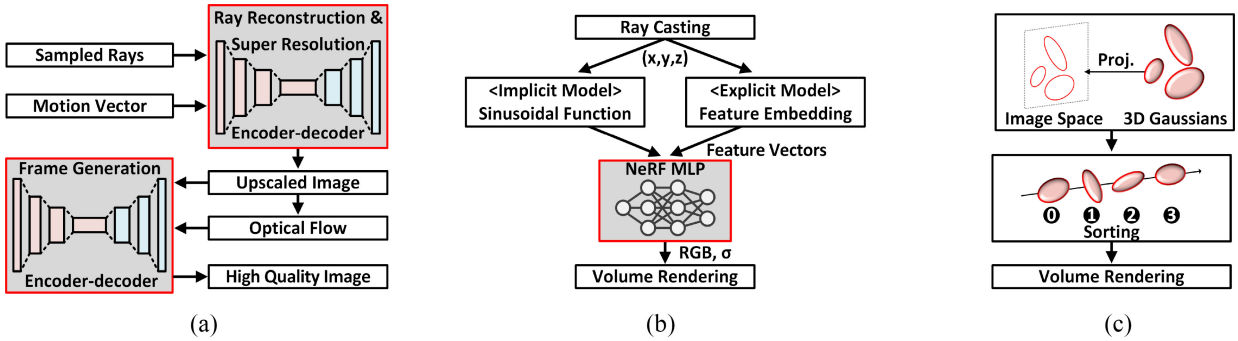

FIGURE 7. Three approaches for AI-based 3-D rendering: (a) DLSS, (b) NeRF, and (c) 3DGS.

consumption. Li et al. [91] analyzed redundant grid size and update frequency of the trainable parameters in hash-based NeRF and found optimal hyperparameters. In the aspect of hardware, it implemented the FF read mapper and BP update merger to improve memory utilization and minimize SRAM writes. This solution achieved 224× speedup for 3-D modeling compared to NVIDIA Jetson Nano [108] with 1.9 W of power consumption. Yuan et al. [98] focused on the ray traversal acceleration by implementing a multistep marching unit that checks for empty space. Designed on a Xilinx FPGA with DDR4 integration, it achieved 2.68× higher throughput compared to NVIDIA Jetson Xavier NX [107], consuming 3.11 W of power.

Besides, Fu et al. [99] accelerated a generalizable NeRF model that does not require per-scene training. This work addressed both memory access for fetching scene features and the large computational cost. It introduced a coarse-then-focus sampling method to reduce the number of samples, and implemented a greedy 3-D point-patch partition to minimize the memory access for scene features. The architectural simulation verified that these optimizations achieved 246× speedup in rendering compared to NVIDIA RTX 2080Ti [109] on the Synthetic NeRF dataset with a power consumption of 9.7 W.

C. 3-D GAUSSIAN SPLATTING ACCELERATOR

3DGS is a recently proposed method for 3-D scene representation that leverages 3-D Gaussians instead of relying on DNN parameters, such as NeRF. This technique encodes the scene using approximately one million 3-D Gaussians, each including trainable parameters, such as shape, color, and opacity. The 3DGS rendering process involves three main stages, as illustrated in Fig. 7(c): 1) projection; 2) sorting; and 3) volume rendering. When user's viewpoint is given, the 3-D Gaussians are mapped onto the 2-D image plane. Subsequently, the Gaussians are sorted based on their depth relative to the camera. The final pixel values are computed using alpha blending, a method similar to NeRF. Although 3DGS benefits from the rasterization operation, which is well optimized for commodity GPUs, it still suffers from low rendering speed (<10 fps) on resource-constrained GPUs.

Lee et al. [100] addressed 3DGS hardware inefficiencies on edge devices, particularly redundant pixel computations and the sorting process. The solution introduced a Gaussian-tile intersection unit to reduce the number of computing tiles and a hierarchical sorting mechanism that performs approximate sorting primarily with lower sorting costs. Simulations of the proposed architecture, running at 1 GHz of clock frequency, demonstrated a 15.86× speedup compared to NVIDIA Jetson Xavier NX [107], with a power consumption of 0.87 W. Feng et al. [101] proposed hybrid rasterizing to replace high-precision rasterization dataflows with a lightweight interpolation of computation results across adjacent pixels. Additionally, it integrated hardware-efficient shape analyzer to eliminate ineffective Gaussian computations. The chip is fabricated in 28-nm process and achieved 373 fps rendering while consuming 664 mW. Song et al. [102] addressed the performance bottleneck caused by irregular off-chip memory access resulting from unordered 3-D Gaussians. This work resolved it by clustering Gaussians based on neighboring surface proximity, enabling streaming DRAM access patterns. Moreover, computed surface normal vectors are utilized to reuse sorting results across frames for latency reduction. The fabricated chip in 28-nm process achieved 142.1 fps and 67.1 iters/s supporting both rendering and modeling.

D. FUTURE DEVELOPMENT DIRECTIONS

The aforementioned rendering technologies have challenges in fully realizing immersive metaverse experiences, particularly on mobile and edge devices. For example, ray tracing is often limited to synthetic or custom environments due to challenges in extracting physical parameters like normal maps from real-world scenes. Similarly, NeRF and 3DGS face challenges in relighting, the process of simulating a scene's appearance under new lighting conditions. As lighting is baked into the model during training, it captures the scene's appearance only under its original lighting, making it difficult to separate appearance from re-render with different light sources. The lack of explicit surface or material representations further complicates this as light interacts directly with surfaces. On the other hand, 3DGS is well suited for high-speed rendering due to its hardware-friendly rasterization operation, while NeRF is appropriate for quick 3-D modeling because of its smaller model size compared to 3DGS. Recently, several algorithm researchers

have explored combining these rendering methods to enhance both image quality and efficiency. For example, Chen et al. [110] fused hash-based NeRF and 3DGS to reduce model (parameter) size while preserving rendering quality. Moenne-Loccoz et al. [111] combined 3DGS with ray tracing to handle secondary lighting effects, such as shadows and reflections. Therefore, future hardware for rendering is expected to integrate several hardware units to accelerate fused rendering pipelines. For example, RT core for ray tracing can be integrated in NeRF/3DGS accelerators to enable secondary lighting effects by accelerating both BVH traversal and rendering pipeline. Moreover, user-interactive rendering [102] will be essential for immersive metaverse experiences, necessitating the integration of additional processing pipelines, such as surface extraction and deformation.

VIII. CONCLUSION

This article provides a detailed survey of hardware architectures and silicon solutions for AI-based 3-D spatial computing systems. As traditional computer vision algorithms transitioned to AI-driven algorithms, hardware architecture designs have also evolved to align with these changes. Future platforms are expected to position AI accelerators as the primary component in an SoC, with auxiliary logic integrated to support specialized tasks. This design approach will enable efficient, versatile, and scalable solutions, meeting the diverse needs of 3-D spatial computing applications across industries. Ultimately, the next generation of 3-D spatial computing platforms will unify various functionalities through AI technologies, paving the way for a more interactive, adaptive, and immersive user experience.

REFERENCES

[1] D. Maturana and S. Scherer, "Voxnet: A 3-D convolutional neural network for real-time object recognition," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2015, pp. 922–928.

[2] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3-D classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 652–660.

[3] M. Kazhdan, T. Funkhouser, and S. Rusinkiewicz, "Rotation invariant spherical harmonic representation of 3-D shape descriptors," in *Proc. Symp. Geometry Process.*, vol. 6, 2003, pp. 156–164.

[4] T. Chen, B. Dai, D. Liu, and J. Song, "Performance of global descriptors for Velodyne-based urban object recognition," in *Proc. IEEE Intell. Veh. Symp.*, 2014, pp. 667–673.

[5] M. De Deuge, A. Quadros, C. Hung, and B. Douillard, "Unsupervised feature learning for classification of outdoor 3-D scans," in *Proc. Aust. Conf. Robot. Autom.*, vol. 2, 2013, pp. 1–8.

[6] G. Chaurasia, A. Nieuwoudt, A.-E. Ichim, R. Szeliski, and A. Sorkine-Hornung, "Passthrough+ real-time stereoscopic view synthesis for mobile mixed reality," *Proc. ACM Comput. Graph. Interact. Techn.*, vol. 3, no. 1, pp. 1–17, 2020.

[7] A. Elluswamy, "Foundation models for autonomy," in *Proc. IEEE/CVF Comput. Vis. Pattern Recognit. Conf. (CVPR) Workshop Auton. Driving*, 2023, pp. 1–9.

[8] Apple. "Apple vision pro." 2025. [Online]. Available: https://www.apple.com/apple-vision-pro/specs/

[9] E. Terry, "Silicon at the heart of HoloLens 2," in *Proc. IEEE Hot Chips 31 Symp. (HCS)*, 2019, pp. 1–26.

[10] Ouster. "Velodyne LiDAR VLS-128." Accessed: Sep. 23, 2024. [Online]. Available: https://data.ouster.io/downloads/datasheets/velodyne/63-9679_Rev-B_DATASHEET_ALPHA-PRIME_web.pdf

[11] Intel. "Intel realsense LiDAR camera L515." Accessed: Apr. 30, 2022. [Online]. Available: https://www.intelrealsense.com/lidar-camera-l515/

[12] K. Grace, J. Salvatier, A. Dafoe, B. Zhang, and O. Evans, "When will AI exceed human performance? Evidence from AI experts," *J. Artif. Intell. Res.*, vol. 62, no. 5, pp. 729–754, 2018.

[13] R. Geirhos et al., "Partial success in closing the gap between human and machine vision," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 23885–23899.

[14] A. Suleiman, Z. Zhang, L. Carlone, S. Karaman, and V. Sze, "Navion: A 2-mW fully integrated real-time visual-inertial Odometry accelerator for autonomous navigation of Nano drones," *IEEE J. Solid-State Circuits*, vol. 54, no. 4, pp. 1106–1119, Apr. 2019.

[15] D. Kanter, *Graphics Processing Requirements for Enabling Immersive VR*, AMD, Santa Clara, CA, USA, 2015.

[16] Y. Deng, Y. Ni, Z. Li, S. Mu, and W. Zhang, "Toward real-time ray tracing: A survey on hardware acceleration and microarchitecture techniques," *ACM Comput. Surveys*, vol. 50, no. 4, pp. 1–41, 2017.

[17] Y. Zhang and T. Funkhouser, "Deep depth completion of a single RGB-D image," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 175–185.

[18] S. Choi, J. Lee, K. Lee, and H.-J. Yoo, "A 9.02 mW CNN-stereo-based real-time 3-D hand-gesture recognition processor for smart mobile devices," in *Proc. IEEE Int. Solid-State Circuits Conf.-(ISSCC)*, 2018, pp. 220–222.

[19] G. Chen et al., "StereoEngine: An FPGA-based accelerator for real-time high-quality stereo estimation with binary neural network," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 11, pp. 4179–4190, Nov. 2020.

[20] Y. Zhang, G. Chen, T. He, Q. Huang, and K. Huang, "ViraEye: An energy-efficient stereo vision accelerator with binary neural network in 55 nm CMOS," in *Proc. 28th Asia South Pac. Design Autom. Conf.*, 2023, pp. 178–179.

[21] Y. Sada, N. Soga, M. Shimoda, A. Jinguji, S. Sato, and H. Nakahara, "Fast monocular depth estimation on an FPGA," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, 2020, pp. 143–146.

[22] D. Im et al., "A mobile 3-D object recognition processor with deep learning-based monocular depth estimation," *IEEE Micro*, vol. 43, no. 3, pp. 74–82, May/Jun. 2023.

[23] W. Park, D. Im, H. Kwon, and H.-J. Yoo, "An efficient unsupervised learning-based monocular depth estimation processor with partial-switchable systolic array architecture in edge devices," in *Proc. IEEE Asian Solid-State Circuits Conf. (A-SSCC)*, 2022, pp. 1–3.

[24] D. Im et al., "DSPU: A 281.6 mW real-time depth signal processing unit for deep learning-based dense RGB-D data acquisition with depth fusion and 3-D bounding box extraction in mobile platforms," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 65, 2022, pp. 510–512.

[25] M. Sun et al., "A 40nm 2TOPS/W depth-completion neural network accelerator SoC with efficient depth engine for realtime LiDAR systems," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 5, pp. 1704–1708, May 2023.

[26] X. Dong, M. A. Garratt, S. G. Anavatti, and H. A. Abbass, "Towards real-time monocular depth estimation for robotics: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 10, pp. 16940–16961, Oct. 2022.

[27] W. Luo, A. G. Schwing, and R. Urtasun, "Efficient deep learning for stereo matching," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 5695–5703.

[28] J. Žbontar and Y. LeCun, "Stereo matching by training a convolutional neural network to compare image patches," *J. Mach. Learn. Res.*, vol. 17, no. 65, pp. 1–32, 2016.

[29] D. Wofk, F. Ma, T.-J. Yang, S. Karaman, and V. Sze, "FastDepth: Fast monocular depth estimation on embedded systems," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, 2019, pp. 6101–6108.

[30] D. Im and H.-J. Yoo, "CamPU: A multi-camera processing unit for deep learning-based 3-D spatial computing systems," in *Proc. 57th IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2024, pp. 50–63.

[31] Y. Feng, B. Tian, T. Xu, P. Whatmough, and Y. Zhu, "Mesorasi: Architecture support for point cloud analytics via delayed-aggregation," in *Proc. 53rd Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2020, pp. 1037–1050.

[32] S. Kim, J. Lee, D. Im, and H.-J. Yoo, "PNNPU: A 11.9 TOPS/W high-speed 3-D point cloud-based neural network processor with block-based point processing for regular DRAM access," in *Proc. Symp. VLSI Circuits*, 2021, pp. 1–2.

[33] Y. Feng, G. Hammonds, Y. Gan, and Y. Zhu, "Crescent: Taming memory irregularities for accelerating deep point cloud analytics," in *Proc. 49th Annu. Int. Symp. Comput. Archit.*, 2022, pp. 962–977.

[34] F. Chen, R. Ying, J. Xue, F. Wen, and P. Liu, "ParallelNN: A parallel Octree-based nearest neighbor search accelerator for 3-D point clouds," in *Proc. IEEE Int. Symp. High Perform. Comput. Architect. (HPCA)*, 2023, pp. 403–414.

[35] P. H. E. Becker, J.-M. Arnau, and A. González, "KD bonsai: ISA-extensions to compress KD trees for autonomous driving tasks," in *Proc. 50th Annu. Int. Symp. Comput. Architect.*, 2023, pp. 1–13.

[36] Z. Ying, S. Bhuyan, Y. Kang, Y. Zhang, M. T. Kandemir, and C. R. Das, "EdgePC: Efficient deep learning analytics for point clouds on edge devices," in *Proc. 50th Annu. Int. Symp. Comput. Architect.*, 2023, pp. 1–14.

[37] C. Chen, X. Zou, H. Shao, Y. Li, and K. Li, "Point cloud acceleration by exploiting geometric similarity," in *Proc. 56th Annu. IEEE/ACM Int. Symp. Microarchit.*, 2023, pp. 1135–1147.

[38] J. Jung et al., "20.6 LSPU: A fully integrated real-time LiDAR-SLAM SoC with point-neural-network segmentation and multi-level kNN acceleration," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 67, 2024, pp. 370–372.

[39] C. Zhou, M. Liu, S. Qiu, X. Cao, Y. Fu, and Y. He, "Sagitta: An energy-efficient sparse 3-D-CNN accelerator for real-time 3-D understanding," *IEEE Internet Things J.*, vol. 10, no. 23, pp. 20703–20717, Dec. 2023.

[40] Q. Cao and J. Gu, "A sparse convolution neural network accelerator for 3-D/4-D point-cloud image recognition on low power mobile device with hopping-index rule book for efficient coordinate management," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, 2022, pp. 106–107.

[41] W. Sun et al., "A 28nm 2-D/3-D unified sparse convolution accelerator with block-wise neighbor searcher for large-scaled voxel-based point cloud network," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2023, pp. 328–330.

[42] X. Feng et al., "A 28-nm energy-efficient sparse neural network processor for point cloud applications using block-wise online neighbor searching," *IEEE J. Solid-State Circuits*, vol. 59, no. 9, pp. 3070–3081, Sep. 2024.

[43] X. Feng, W. Sun, X. Lin, S. Fan, H. Yang, and Y. Liu, "A 28nm 1.2 GHz 5.27 TOPS/W scalable vision/point cloud deep fusion processor with CAM-based universal mapping unit for BEVFusion applications," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, 2024, pp. 1–2.

[44] X. Feng, X. Lin, H. Yang, Y. Liu, and W. Sun, "A scalable BEV perception processor for image/point cloud fusion applications using CAM-based universal mapping unit," *IEEE J. Solid-State Circuits*, vol. 60, no. 3, pp. 1002–1013, Mar. 2025.

[45] M. Lee et al., "SPADE: Sparse pillar-based 3-D object detection accelerator for autonomous driving," in *Proc. IEEE Int. Symp. High Perform. Comput. Architect. (HPCA)*, 2024, pp. 454–467.

[46] S. Lim, J. Heo, J. Yang, and J.-Y. Kim, "A 38.5 TOPS/W point cloud neural network processor with virtual pillar and Quadtree-based workload management for real-time outdoor BEV detection," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, 2024, pp. 1–2.

[47] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "PointNet++: Deep hierarchical feature learning on point sets in a metric space," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, p. 15.

[48] C. R. Qi, O. Litany, K. He, and L. J. Guibas, "Deep hough voting for 3-D object detection in point clouds," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 9277–9286.

[49] R. Bridson, "Fast Poisson disk sampling in arbitrary dimensions," *SIGGRAPH Sketches*, vol. 10, no. 1, p. 1, 2007.

[50] D. Tran, L. Bourdev, R. Fergus, L. Torresani, and M. Paluri, "Learning spatiotemporal features with 3-D convolutional networks," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2015, pp. 4489–4497.

[51] B. Graham, "Sparse 3-D convolutional neural networks," 2015, *arXiv:1505.02890*.

[52] B. Graham and L. Van der Maaten, "Submanifold sparse convolutional networks," 2017, *arXiv:1706.01307*.

[53] C. Choy, J. Gwak, and S. Savarese, "4D spatio-temporal ConvNets: Minkowski convolutional neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 3075–3084.

[54] Y. Yan, Y. Mao, and B. Li, "Second: Sparsely embedded convolutional detection," *Sensors*, vol. 18, no. 10, p. 3337, 2018.

[55] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2012, pp. 3354–3361.

[56] G. Shi, R. Li, and C. Ma, "PillarNet: Real-time and high-performance pillar-based 3-D object detection," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 35–52.

[57] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, "Pointpillars: Fast encoders for object detection from point clouds," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 12697–12705.

[58] Y. Lin, Z. Zhang, H. Tang, H. Wang, and S. Han, "PointACC: Efficient point cloud accelerator," in *Proc. 54th Annu. IEEE/ACM Int. Symp. Microarchitect. (MICRO)*, 2021, pp. 449–461.

[59] D. Im, S. Kang, D. Han, S. Choi, and H.-J. Yoo, "A 4.45 ms low-latency 3-D point-cloud-based neural network processor for hand pose estimation in immersive wearable devices," in *Proc. IEEE Symp. VLSI Circuits*, 2020, pp. 1–2.

[60] Y. Kim, J. So, C. Hwang, W. Cheng, J. Choi, and J. H. Ko, "An FPGA-based energy-efficient real-time hand pose estimation system with an integrated image signal processor for indirect 3-D time-of-flight sensors," *IEEE Internet Things J.*, vol. 12, no. 2, pp. 1817–1830, Jan. 2025.

[61] Y. Lu et al., "9.7 a 184 μ W real-time hand-gesture recognition system with hybrid tiny classifiers for smart wearable devices," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 64, 2021, pp. 156–158.

[62] Y. Lu, Z. Li, Y. Chen, and T. T.-H. Kim, "A 181μW real-time 3-D hand gesture recognition system based on bi-directional convolution and computing-efficient feature clustering," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, 2022, pp. 1–2.

[63] B. Stenger, A. Thayananthan, P. H. Torr, and R. Cipolla, "Model-based hand tracking using a hierarchical Bayesian filter," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 28, no. 9, pp. 1372–1384, Sep. 2006.

[64] C. Qian, X. Sun, Y. Wei, X. Tang, and J. Sun, "Realtime and robust hand tracking from depth," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2014, pp. 1106–1113.

[65] I. Oikonomidis et al., "Efficient model-based 3-D tracking of hand articulations using Kinect," in *Proc. BmVC*, vol. 1, 2011, p. 3.

[66] L. Ge, Y. Cai, J. Weng, and J. Yuan, "Hand PointNet: 3-D hand pose estimation using point sets," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8417–8426.

[67] W. Cheng, J. H. Park, and J. H. Ko, "HandFoldingNet: A 3-D hand pose estimation network using multiscale-feature guided folding of a 2-D hand skeleton," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 11260–11269.

[68] G. Moon, J. Y. Chang, and K. M. Lee, "V2v-PoseNet: Voxel-to-Voxel prediction network for accurate 3-D hand and human pose estimation from a single depth map," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 5079–5088.

[69] L. Ge, Z. Ren, and J. Yuan, "Point-to-point regression PointNet for 3-D hand pose estimation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 475–491.

[70] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.

[71] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI*, vol. 1, no. 8, p. 9, 2019.

[72] G. Team et al., "Gemini: A family of highly capable multimodal models," 2023, *arXiv:2312.11805*.

[73] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[74] Z. Li et al., "An 879GOPS 243mW 80fps VGA fully visual CNN-SLAM processor for wide-range autonomous exploration," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2019, pp. 134–136.

[75] J. Jung et al., "An energy-efficient processor for real-time semantic LiDAR SLAM in mobile robots," *IEEE J. Solid-State Circuits*, vol. 60, no. 1, pp. 112–124, Jan. 2025.

[76] M. Li et al., "SLAM-CIM: A visual SLAM backend processor with dynamic-range-driven-skipping linear-solving FP-CIM macros," *IEEE J. Solid-State Circuits*, vol. 59, no. 11, pp. 3853–3865, Nov. 2024.

[77] G. Park et al., "20.8 space-mate: A 303.5mW real-time sparse mixture-of-experts-based NeRF-SLAM processor for mobile spatial computing," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 67, 2024, pp. 374–376.

[78] L. Wu, H. Zhu, S. He, J. Zheng, C. Chen, and X. Zeng, "GauSPU: 3-D Gaussian splatting processor for real-time SLAM systems," in *Proc. 57th IEEE/ACM Int. Symp. Microarchitect. (MICRO)*, 2024, pp. 1562–1573.

[79] D. Li et al., "DXSLAM: A robust and efficient visual SLAM system with deep features," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2020, pp. 4958–4965.

[80] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison, "iMAP: Implicit mapping and positioning in real-time," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 6229–6238.

[81] Z. Zhu et al., "NICE-SLAM: Neural implicit scalable encoding for SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 12776–12786.

[82] G. Klein and D. Murray, "Parallel tracking and mapping for small AR workspaces," in *Proc. 6th IEEE ACM Int. Symp. Mixed Augmented Real.*, 2007, pp. 225–234.

[83] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM: A versatile and accurate monocular SLAM system," *IEEE Trans. Robot.*, vol. 31, no. 5, pp. 1147–1163, Oct. 2015. [Online]. Available: http://dx.doi.org/10.1109/TRO.2015.2463671

[84] B. Bescos, J. M. Facil, J. Civera, and J. Neira, "DynaSLAM: Tracking, mapping, and in-painting in dynamic scenes," *IEEE Robot. Autom. Lett.*, vol. 3, no. 4, pp. 4076–4083, Oct. 2018. [Online]. Available: http://dx.doi.org/10.1109/LRA.2018.2860039

[85] C. Cadena et al., "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Trans. Robot.*, vol. 32, no. 6, pp. 1309–1332, Dec. 2016.

[86] Y. Jeong, D. Nister, D. Steedly, R. Szeliski, and I.-S. Kweon, "Pushing the envelope of modern methods for bundle adjustment," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 8, pp. 1605–1617, Aug. 2012.

[87] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vis.*, vol. 60, no. 2, pp. 91–110, Nov. 2004. [Online]. Available: https://doi.org/10.1023/B:VISI.0000029664.99615.94

[88] N. Keetha et al. "SplaTAM: Splat, track & map 3-D Gaussian's for dense RGB-D SLAM." 2024. [Online]. Available: https://arxiv.org/abs/2312.02126

[89] G. Park et al., "An artificial-intelligence-based SLAM processor with scene-adaptive sampling and hybrid NeRF model training acceleration," *IEEE Trans. Circuits Syst. Artif. Intell.*, vol. 1, no. 1, pp. 3–14, Sep. 2024.

[90] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," in *Proc. ECCV*, 2020, pp. 405–421.

[91] S. Li et al., "Instant-3-D: Instant neural radiance field training towards on-device AR/VR 3-D reconstruction," in *Proc. 50th Annu. Int. Symp. Comput. Architect. (ISCA)*, 2023, p. 5. [Online]. Available: https://doi.org/10.1145/3579371.3589115

[92] "NVIDIA DLSS." Accessed: Oct. 22, 2024. [Online]. Available: https://www.nvidia.com/en-us/geforce/news/nvidia-dlss-3-5-ray-reconstruction/

[93] D. Han, J. Ryu, S. Kim, S. Kim, and H.-J. Yoo, "2.7 MetaVRain: A 133mW real-time hyper-realistic 3-D-NeRF processor with 1-D–2-D hybrid-neural engines for metaverse on mobile devices," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2023, pp. 50–52.

[94] W. Jo et al., "NeRPIM: A 4.2 mJ/frame neural rendering processing-in-memory processor with space encoding block-wise mapping for mobile devices," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, 2023, pp. 1–2.

[95] J. Park, D. Han, J. Ryu, D. Im, G. Park, and H.-J. Yoo, "A 33.6 FPS embedding based real-time neural rendering accelerator with switchable computation skipping architecture on edge device," in *Proc. IEEE Asian Solid-State Circuits Conf. (A-SSCC)*, 2023, pp. 1–3.

[96] J. Ryu et al., "20.7 NeuGPU: A 18.5mJ/iter neural-graphics processing unit for instant-modeling and real-time rendering with segmented-hashing architecture," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 67, 2024, pp. 372–374.

[97] J. Lee, K. Choi, J. Lee, S. Lee, J. Whangbo, and J. Sim, "NeuRex: A case for neural rendering acceleration," in *Proc. 50th Annu. Int. Symp. Comput. Architect. (ISCA)*, 2023, p. 4. [Online]. Available: https://doi.org/10.1145/3579371.3589056

[98] Z. Yuan et al., "A 0.59uJ/pixel high-throughput energy-efficient neural volume rendering accelerator on FPGA," in *Proc. IEEE CICC*, Apr. 2024, pp. 1–2.

[99] Y. Fu et al., "Gen-NeRF: Efficient and Generalizable neural radiance fields via algorithm-hardware co-design," in *Proc. 50th Annu. Int. Symp. Comput. Architect. (ISCA)*, 2023, pp. 1–12. [Online]. Available: https://doi.org/10.1145/3579371.3589109

[100] J. Lee, S. Lee, J. Lee, J. Park, and J. Sim, "GSCore: Efficient radiance field rendering via architectural support for 3-D Gaussian Splatting," in *Proc. 29th ACM Int. Conf. Architect. Support Program. Lang. Oper. Syst.* 2024, pp. 497–511. [Online]. Available: https://doi.org/10.1145/3620666.3651385

[101] X. Feng, H. Wang, C. Tang, T. Wu, H. Yang, and Y. Liu, "1.78mJ/frame 373fps 3-D GS processor based on shape-aware hybrid architecture using earlier computation skipping and Gaussian cache scheduler," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 68, 2025, pp. 1–3.

[102] S. Song et al., "IRIS: An 8.55mJ/frame spatial computing SoC for interactable rendering and surface-aware modeling with 3-D Gaussian splatting," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2025, pp. 1–8.

[103] "NVIDIA Ada lovelace." Accessed: Oct. 22, 2024. [Online]. Available: https://www.nvidia.com/en-us/geforce/ada-lovelace-architecture/

[104] NVIDIA DLSS performance." Accessed: Oct. 22, 2024. [Online]. Available: https://www.nvidia.com/en-us/geforce/news/dlss-3-ghost-of-tsushima/

[105] A. Chen, Z. Xu, A. Geiger, J. Yu, and H. Su, "TensoRF: Tensorial radiance fields," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2022, pp. 333–350.

[106] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Trans. Graph.*, vol. 41, no. 4, pp. 1–15, Jul. 2022. [Online]. Available: https://doi.org/10.1145/3528223.3530127

[107] "NVIDIA Jetson xavier NX." Accessed: Oct. 22, 2024. [Online]. Available: https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-xavier-nx/

[108] "NVIDIA jetson nano." Accessed: Oct. 22, 2024. [Online]. Available: https://developer.nvidia.com/embedded/jetson-nano

[109] "NVIDIA RTX 2080 TI." Accessed: Oct. 22, 2024. [Online]. Available: https://www.nvidia.com/en-me/geforce/graphics-cards/rtx-2080-ti/

[110] Y. Chen, Q. Wu, W. Lin, M. Harandi, and J. Cai, "HAC: Hash-grid assisted context for 3-D Gaussian splatting compression," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 1–17.

[111] N. Moenne-Loccoz et al., "3-D Gaussian ray tracing: Fast tracing of particle scenes," *ACM Trans. Graph.*, vol. 43, no. 6, pp. 1–19, 2024.

DONGSEOK IM received the B.S. degree in electrical engineering from the Pohang University of Science and Technology, Pohang, South Korea, in 2018, and the M.S. and Ph.D. degrees in electrical engineering from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2020 and 2023, respectively.

He was a Postdoctoral Researcher with the Institute of Information and Electronics, KAIST from 2023 to 2025. He is currently working with Google, San Diego, CA, USA, as an AI/ML System and Silicon Architect. His research interests include energy-efficient AI/ML system-on-chip design; digital circuit-, architecture-, and software-level AI/ML accelerator design; and custom silicon chip and demonstration system board development.

GWANGTAE PARK (Graduate Student Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electrical engineering from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2019, 2021, and 2024, respectively.

He is currently a Postdoctoral Researcher with KAIST. His current research interests include AI/ML hardware acceleration—specifically addressing sparsity, quantization, memory system design, and hardware–software co-design. He also focuses on domain-specific hardware for 2-D/3-D vision processing and custom digital ASIC design.

JUNHA RYU (Graduate Student Member, IEEE) received the B.S., M.S., and Ph.D. degrees from the School of Electrical Engineering, Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2020, 2022, and 2025, respectively.

He is currently a Postdoctoral Researcher with Information and Electronics Research Institute, KAIST. His current research focuses on deep neural network accelerators, hardware-friendly computer vision algorithms, and energy-efficient system-on-chip design, with a special interest in AI-powered AR/VR and 3-D vision systems.

HOI-JUN YOO (Fellow, IEEE) graduated from the Electronic Department of Seoul National University, Seoul, Korea, in 1983. He received the M.S. and Ph.D degrees in electrical engineering from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, in 1985 and 1988, respectively.

In 1998, he joined the faculty of the Department of Electrical Engineering, KAIST. He is currently an ICT Chair Professor with the School of Electrical Engineering, Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, where he is also the Director of the System Design Innovation and Application Research Center, PIM Semiconductor Design Research Center (AI-PIM), and the Institute of Information Technology Convergence. He is also the Dean of the Graduate School of AI Semiconductor, KAIST. More details are available at http://ssl.kaist.ac.kr.

Dr. Yoo served as a member of the Executive Committee for the International Solid-State Circuits Conference (ISSCC), the Symposium on Very Large-Scale Integration, and the Asian Solid-State Circuits Conference (A-SSCC), the TPC Chair for the A-SSCC 2008 and the International Symposium on Wearable Computer 2010, the IEEE Distinguished Lecturer from 2010 to 2011, the Far East Chair for the ISSCC from 2011 to 2012, the Technology Direction Sub-Committee Chair for the ISSCC in 2013, the TPC Vice-Chair for the ISSCC in 2014, and the TPC Chair for the ISSCC in 2015.